

Laboratory Work 1

ERD Diagram

International Airport Database

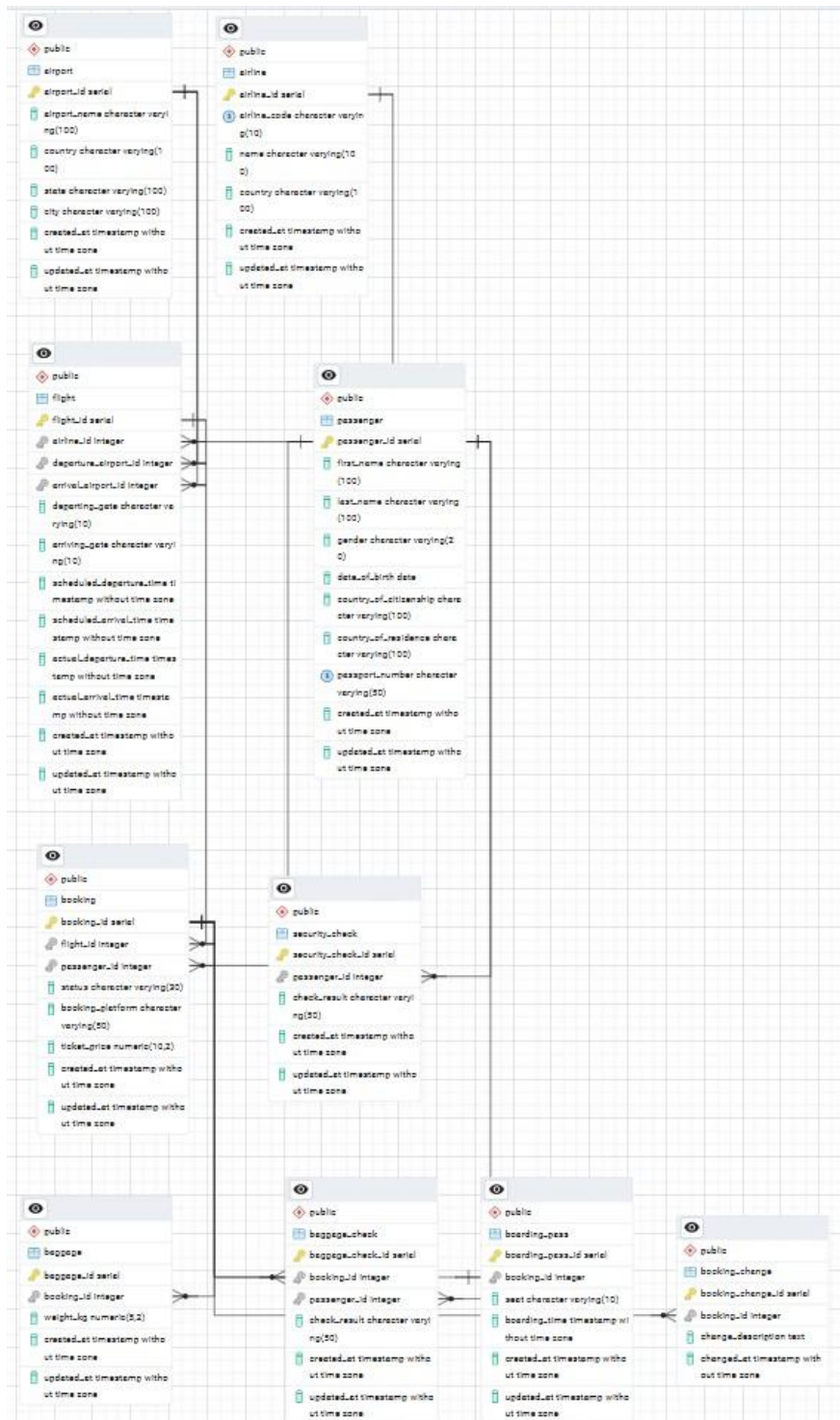
Student: Amanzhol Alizhan

Course: Databases

Submission: PDF Document with ER-Diagram and Description

ER Diagram (pgAdmin 4)

The diagram below was generated in pgAdmin 4's ERD Tool from the **international_airport_db** PostgreSQL database, showing all 10 entities, their attributes, primary keys, foreign keys, and relationships.



List of Entities and Attributes

Entity: airport

Attribute	Data Type	Key
airport_id	SERIAL	PK
airport_name	VARCHAR(100)	
country	VARCHAR(100)	

state	VARCHAR(100)	
city	VARCHAR(100)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Attribute	Data Type	Key
passenger_id	SERIAL	PK

first_name	VARCHAR(100)	
------------	--------------	--

Entity: airline

Attribute	Data Type	Key
airline_id	SERIAL	PK
airline_code	VARCHAR(10)	UNIQUE
name	VARCHAR(100)	
country	VARCHAR(100)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Entity: flight

Attribute	Data Type	Key
flight_id	SERIAL	PK
airline_id	INTEGER	FK → airline
departure_airport_id	INTEGER	FK → airport
arrival_airport_id	INTEGER	FK → airport
departing_gate	VARCHAR(10)	
arriving_gate	VARCHAR(10)	
scheduled_departure_time	TIMESTAMP	
scheduled_arrival_time	TIMESTAMP	
actual_departure_time	TIMESTAMP	
actual_arrival_time	TIMESTAMP	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Entity: passenger

last_name	VARCHAR(100)	
gender	VARCHAR(20)	
date_of_birth	DATE	
country_of_citizenship	VARCHAR(100)	

country_of_residence	VARCHAR(100)	
passport_number	VARCHAR(50)	UNIQUE
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Entity: booking

Attribute	Data Type	Key
booking_id	SERIAL	PK
flight_id	INTEGER	FK → flight
passenger_id	INTEGER	FK → passenger
status	VARCHAR(20)	
booking_platform	VARCHAR(50)	
ticket_price	NUMERIC(10,2)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Entity: booking_change

Attribute	Data Type	Key
booking_change_id	SERIAL	PK
booking_id	INTEGER	FK → booking
change_description	TEXT	
changed_at	TIMESTAMP	

Entity: boarding_pass

Attribute	Data Type	Key
boarding_pass_id	SERIAL	PK
booking_id	INTEGER	FK → booking
seat	VARCHAR(10)	
boarding_time	TIMESTAMP	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Entity: baggage

Attribute	Data Type	Key
baggage_id	SERIAL	PK
booking_id	INTEGER	FK → booking
weight_kg	NUMERIC(5,2)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Entity: baggage_check

Attribute	Data Type	Key
baggage_check_id	SERIAL	PK
booking_id	INTEGER	FK → booking
passenger_id	INTEGER	FK → passenger
check_result	VARCHAR(50)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Entity: security_check

Attribute	Data Type	Key
security_check_id	SERIAL	PK
passenger_id	INTEGER	FK → passenger
check_result	VARCHAR(50)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Relationships and Cardinalities

The table below lists every foreign-key relationship in the database, together with its cardinality and participation, expressed using crow's-foot notation as shown in the ER diagram (one bar = 'one', crow's foot = 'many').

Parent (1)	Child (Many)	Cardinality	Description
airport	flight (departure_airport_id)	1 : M	One airport is the departure point for many flights.
airport	flight (arrival_airport_id)	1 : M	One airport is the arrival point for many flights.
airline	flight	1 : M	One airline operates many flights.
flight	booking	1 : M	One flight can have many bookings.
passenger	booking	1 : M	One passenger can make many bookings.
booking	booking_change	1 : M	One booking can have many recorded changes.
booking	boarding_pass	1 : M	One booking is associated with boarding pass(es).
booking	baggage	1 : M	One booking can include several baggage items.
booking	baggage_check	1 : M	One booking's baggage can undergo multiple checks.
passenger	baggage_check	1 : M	One passenger's baggage checks are tracked over time.
passenger	security_check	1 : M	One passenger can pass through multiple security checks.

Participation constraints: Every child-side relationship above is mandatory for the child (a booking cannot exist without a flight and a passenger, a boarding pass cannot exist without a booking, etc.), while participation is optional on the parent side (e.g., an airport may exist with zero flights, a passenger may exist with zero bookings).

Note: Many-to-many relationships are not present in this schema. All relationships are One-to-Many, implemented through foreign key columns on the 'many' side referencing the primary key of the 'one' side.

SQL Implementation Description

The schema was implemented in PostgreSQL 18 inside the **international_airport_db** database, using pgAdmin 4 as the administration and ERD design tool. The implementation follows these principles:

- **Primary keys:** Every table uses a SERIAL (auto-incrementing integer) column named <entity>_id as its primary key, guaranteeing a unique identifier for each row.

- **Foreign keys:** Referential integrity is enforced with FOREIGN KEY constraints. For example, `flight.airline_id REFERENCES airline(airline_id)`, and `booking.passenger_id REFERENCES passenger(passenger_id)`.
- **Data types:** Text data uses VARCHAR with explicit length limits (e.g., `VARCHAR(100)` for names), free-form text uses TEXT (`change_description`), money values use NUMERIC(10,2) for exact precision, and all date/time attributes use `TIMESTAMP WITHOUT TIME ZONE`.
- **Auditing columns:** Almost every table includes `created_at` and `updated_at` `TIMESTAMP` columns to track record creation and modification, which is standard practice for operational databases.
- **Normalization:** The schema is normalized to at least Third Normal Form (3NF) — each non-key attribute depends only on its table's primary key, and repeating groups (such as multiple baggage items or multiple booking changes per booking) are placed in their own child tables rather than duplicated within booking.
- **Tooling:** Tables were created with standard `CREATE TABLE` statements in pgAdmin's Query Tool, and the ERD Tool was then used to reverse-engineer and visualize the resulting schema, including automatically detected primary key and foreign key relationships.

Example CREATE TABLE statement (flight):

```
CREATE TABLE flight (
    flight_id SERIAL PRIMARY KEY,
    airline_id INTEGER REFERENCES airline(airline_id),
    departure_airport_id INTEGER REFERENCES airport(airport_id),
    arrival_airport_id INTEGER REFERENCES airport(airport_id),
    departing_gate VARCHAR(10), arriving_gate VARCHAR(10),
    scheduled_departure_time TIMESTAMP, scheduled_arrival_time
    TIMESTAMP, actual_departure_time TIMESTAMP,
    actual_arrival_time TIMESTAMP, created_at TIMESTAMP,
    updated_at TIMESTAMP
);
```

Conclusion

In this laboratory work, an Entity-Relationship Diagram was designed and implemented for an International Airport Database. Ten entities were identified from the system description: `airport`, `airline`, `flight`, `passenger`, `booking`, `booking_change`, `boarding_pass`, `baggage`, `baggage_check`, and `security_check`.

For each entity, attributes were determined together with appropriate data types, primary keys, and foreign keys needed to preserve data integrity. Relationships between entities were analyzed and modeled as One-to-Many associations with clear cardinality and participation constraints, for example: one airline operates many flights, one flight can have many bookings, and one booking can be linked to several baggage items and baggage checks.

The diagram was built and verified using pgAdmin 4's ERD Tool, connected to a live PostgreSQL 18 database, which allowed the schema to be created, visualized, and cross-checked directly against the running tables and constraints.

This exercise reinforced the practical value of ER modeling before writing SQL: identifying entities and relationships early makes it much easier to enforce data integrity through foreign keys, avoid data duplication through normalization, and design a schema capable of representing real-world

operational processes — such as passenger bookings, flight scheduling, and airport security procedures — in a structured and reliable way.